# AlgebraicPowerSeries documentation

Noé Terrien

July 3, 2026

## Contents


**1 Introduction** **2**
- 1.1 Objectives . . . 2
- 1.2 Julia version . . . 2
- 1.3 Dependencies . . . 2
- 1.4 Examples . . . 3
- 1.5 Benchmarks . . . 4

**2 User guide** **4**
- 2.1 PowerSeries . . . 4
- 2.2 TaylorExpansionSeries . . . 5
- 2.3 TranslatedSeries . . . 5
- 2.4 PDESeries and LocalizedPDESeries . . . 6
  - 2.4.1 The "self" series . . . 6
  - 2.4.2 Writing the partial differential equation . . . 7
  - 2.4.3 An important note on the use of operators with series that do not have the same variables or centers . . . 8
  - 2.4.4 Constructing PDESeries and LocalizedPDESeries . . . 10
  - 2.4.5 An example . . . 10

**3 Detailed documentation** **11**
- 3.1 PowerSeries . . . 11
  - 3.1.1 PowerSeries attributes . . . 11
  - 3.1.2 ScalarSeriesSymbol . . . 12
  - 3.1.3 SeriesCoefficient . . . 13
  - 3.1.4 Coefficients indexing : lin, trunc, fullsym . . . 13
- 3.2 Series defined by Partial Differential Equations . . . 14
  - 3.2.1 SymbolicSeries, EvaluatedSymbolicSeries and SymbolicSeriesEquation . . 14
  - 3.2.2 NlinearSeriesOperation . . . 15
  - 3.2.3 MultilinearSeriesOperation . . . 16
  - 3.2.4 getindex method on SymbolicSeries . . . 17
  - 3.2.5 getNum and getSymbolics method . . . 18
  - 3.2.6 The compute_coefficients! method . . . 19

**4 Appendix** **19**
- 4.1 Evaluating a SymbolicSeries . . . 19
  - 4.1.1 $y_D = c_D$ . . . 20

4.1.2 $y_D \neq c_D$ is a constant . . . 20
4.1.3 $y_D$ is a variable and $y_D \notin \{y_1, \ldots, y_{D-1}\}$ . . . 20
4.1.4 $y_D$ is a variable and $\exists k \in \{1, \ldots, D-1\}, y_k = y_D$ . . . 20
4.2 Operations defined on SymbolicSeries . . . 21
4.2.1 Translation of a SymbolicSeries . . . 21
4.2.2 Multiplication of two SymbolicSeries . . . 22
4.3 Operations defined on EvaluatedSymbolicSeries . . . 23
4.3.1 Differentiation of an EvaluatedSymbolicSeries . . . 23
4.3.2 Integration of EvaluatedSymbolicSeries . . . 23

# 1 Introduction

## 1.1 Objectives

This module introduces an algebraic representation of potentially infinite power series and arrays of power series of any number of variables. These series are represented by PowerSeries objects, which contain the instructions to compute the coefficients of the series up to an arbitrarily high order (though that may come at the cost of some algorithmic complexity).

It was primarily developped to compute backstepping kernels (see [1] for details on this method), but it should technically work to solve a number of well-posed Partial Differential Equations. It was tested on multiple examples (see Examples section), correctly reproducing results from matematica but possible bugs may remain. Users are encouraged to do their own tests to verify that everything works as expected for their personal use.

## 1.2 Julia version

This module was developped in Julia 1.12.6

## 1.3 Dependencies

This module makes use of several pre-existing Julia modules:

- Symbolics: v7.24.2
- TaylorSeries: v0.21.10
- LinearSolve: v3.85.2

These can be installed by typing `]` in the Julia REPL and then using the command
`add Symbolics TaylorSeries LinearSolve`

Examples use Pluto notebooks and GLMakie for display

- Pluto: v0.20.27
- PlutoUI: v0.7.83
- GLMakie: v0.13.10

These can be installed by typing `]` in the Julia REPL and then using the command
`add Pluto PlutoUI GLMakie`

Tests and benchmarking rely on Julia Test and BenchmarkTools modules

- Test: v1.11.0
- BenchmarkTools: v1.8.0

These can be installed by typing `]` in the Julia REPL and then using the command `add Test BenchmarkTools`

## 1.4 Examples

Several examples are provided in the form of Pluto notebooks in the "examples" folder. To start Pluto, run `import Pluto` and then `Pluto.run()` in your REPL. Then open the notebook you want. Alternatively, you can also run the "<example>.jl" files directly.

These examples were used to assess the reliability of the module by reproducing the results from matematic notebooks presented in [2]

| <example>.jl | PDE solved |
|---|---|
| 1-D reaction diffusion equation with space-varying reaction | Example 1 : $$\begin{cases} K_{xx}(x,\xi) - K_{\xi\xi} = \frac{\lambda(\xi)+c}{\varepsilon} K(x,\xi) \\ K(x,x) = -\frac{1}{2\varepsilon}\int_0^x (\lambda(\xi)+c)d\xi \\ K(x,0) = 0 \end{cases}$$ |
| coupled 1D kernel | Example 4 : $$\begin{cases} \mu(x)K_x^{vv} + \mu(\xi)K_\xi^{vv} = -\mu'(\xi)K^{vv} + c_2(\xi)K^{vu} + [c_4(x) - c_4(\xi)]K^{vu} \\ \mu(x)K_x^{vu} - \varepsilon(\xi)K_\xi^{vu} = -\varepsilon'(\xi)K^{vu} + c_3(\xi)K^{vv} + [c_4(x) - c_1(\xi)]K^{vv} \\ K^{vv}(x,0) = \frac{q\varepsilon(0)}{\mu(0)}K^{vu}(x,0) \\ (\varepsilon(x)+\mu(x))K^{vu}(x,x) = -c_3(x) \end{cases}$$ |
| 2x2 1-D linear hyperbolic system | Inspired from [3] $$\begin{cases} \Sigma(x)K_x(x,y) + K_y(x,y)\Sigma(y) = K(x,y)[C(y) - \Sigma'(y)] - C_0(x)K(x,y) \\ K(x,x)\Sigma(x) - \Sigma(x)K(x,x) = C(x) - C_0(x) \\ K(x,0)\Sigma(0)\begin{pmatrix} q \\ 1 \end{pmatrix} = 0 \end{cases}$$ where $$\Sigma = \begin{pmatrix} -\varepsilon_1 & 0 \\ 0 & \varepsilon_2 \end{pmatrix}$$ with $\varepsilon_1, \varepsilon_2 > 0$ and $$C = \begin{pmatrix} c_1 & c_2 \\ c_3 & c_4 \end{pmatrix} \text{ and } C_0 = \begin{pmatrix} c_1 & 0 \\ 0 & c_4 \end{pmatrix}$$ |
| localized power series - 1-D reaction diffusion equation | Illustrates the use of LocalizedPDESeries with Example 1 |
| fixing divergence in 1-D reaction diffusion equation with space-varying reaction | Illustrates the use of TranslatedSeries to avoid using LocalizedPDESeries using divergent Example 1. Includes a benchmark to compare performance differences |

## 1.5 Benchmarks

Some benchmarks will be proposed in the future to compare different solvers for PDESeries that are hardware-dependent or trying to use sparsity

# 2 User guide

This user guide explains how to use the objects defined in "AlgebraicPowerSeries.jl" and "utilitaries.jl" files to expand functions into their Taylor series, solve Partial Differential Equations (PDE) and build the resulting functions' approximations.

## 2.1 PowerSeries

`PowerSeries{T,D}` is an abstract type representing power series, or to be more precise, an `Array{D}` of power series. The common interface accross every concrete `PowerSeries` allows them to perform a number of operations:

- `compute_coefficients!` – PowerSeries is an interface to represent abstract power series. However, most uses require accessing its coefficients. To do so, these coefficients must first be computed up to a given order N by using `compute_coefficients!(ps, N)` where ps is a PowerSeries.
- `ps.coefficients` – The first way of accesing the previously computed coefficients is to access the `coefficients` attribute of the PowerSeries. This array is a `Array{D, Vector{T}}`. Since a PowerSeries represents in reality an array of scalar PowerSeries, to access the coefficients this way, one must first access the corresponding scalar series. The coefficients are then sorted in increasing order of their coefficients (i.e : $a_1 + a_2x + a_3y + a_4x^2 + a_5xy + a_6y^2 + ...$). For instance, one could do `ps.coefficients[1,2][3]` to access the coefficient of $y$ in a two variables series $x, y$ at the scalar index $(1, 2)$
- `getindex` – The `getindex` method is another way of accessing a series coefficients. Calling for instance `ps[1,2]` will return a `ScalarSeriesSymbol` that can then be called using the `getindex` method an additionnal time which will return a `SeriesCoefficient`. Indexing a ScalarSeriesSymbol requires using the truncated indices, i.e the series is indexed as $a_{0,0} + a_{1,0}x + a_{1,1}y + a_{2,0}x^2 + a_{2,1}xy + a_{2,2}y^2 + ...$ (see Coefficients indexing : lin, trunc, fullsym for details on the different ways of indexing). Calling for instance `ps[1,2][1,1]` will return the corresponding `SeriesCoefficient`. Using then `getNum` on this `SeriesCoefficient` will return its value if it has already been computed and throw an error otherwise. Thus, `getNum(ps[1,2][1,1])` is the same as `ps.coefficients[1,2][3]`. See more details on ScalarSeriesSymbol and SeriesCoefficient in the detailed documentation.
- `build_matrix_elt` – Given a `PowerSeries ps`, it is possible to retrieve the function it represents (or rather its polynomial approximation). To do so, one can use `build_matrix_elt(ps)`. This will return an `Array{D}` of functions of the `PowerSeries`' variables. Additionnaly, one can eventually provide a second argument N to only compute the polynomial function up to degree N. If N is greater than the order up to which the series has been computed, an error will be thrown
- `build` – This works similarly to `build_matrix_elt` but instead of returning an array of functions, it returns a function that returns arrays. However, this seems to be less performant and the returned functions takes longer to evaluate

Most PowerSeries require a type parameter T to be constructed, which is the type the coefficients will be stored as. Some PowerSeries will have other uses for it. T must be a concrete type.

In addition, each PowerSeries as an ID, stored as the `seriesID::Symbol` attribute. This ID should be different for all series and is used to generate a unique representation of each of the series' coefficients.

### 2.2 TaylorExpansionSeries

`TaylorExpansionSeries{T,D}` is a subtype of `PowerSeries{T,D}` used to represent a power series that is the Taylor expansion of a function $f : \mathbb{R}^m \longrightarrow \mathbb{R}^{n_1 \times n_2 \times \cdots \times n_D}$

To create a `TaylorExpansionSeries`, one will need to provide several arguments to its constructor :

- The coefficients' type `T`
- A `seriesID`
- Its `variables`, which must come from the "Symbolics" package. They can be created using `@variables x y z` for instance and must be given as a `VectorNum`
- The function it represents. This must be an `Array{D}` of expressions that use the previously provided variables
- The `center` of the series, as a `Vector` that has the same length as the `variables` Vector.

Here is an example of how to use this constructor to developp the *sin* function around 0:

```
sin_ps = TaylorExpansionSeries{Float64}(:sin, [x], [sin(x)], [0])
```

One can then use it like any other `PowerSeries`, by first computing its coefficients up to a given order N:

```
compute_coefficients!(sin_ps, 30)
```

In the background, this method computes the coefficients using the `TaylorSeries` Julia module, allowing it to very rapidly compute the Taylor expansion of the function.

### 2.3 TranslatedSeries

Sometimes, one might want to change a `PowerSeries`' center. For instance, this can, in some cases, change the radius of convergence of the series. This is done using a `TranslatedSeries`. Its constructor is pretty simple :

```
TranslatedSeries(seriesID::Symbol, ref::PowerSeries, new_center::Vector)
```

This object stores a referenced `PowerSeries` ref and a new center to be able to compute the coefficients of the series around the new center, though at the cost of a loss of precision.

For the `compute_coefficients!` method to work in a mathematically correct way, there must exist a continuous path between both centers, inside the domain of holomorphy of the function represented by the `ref` series. The formula that is used can be deduced from Cauchy's integral formula (see [4]) which gives :

**Proposition :** If $f$ is holomorphic on some connected open set $\Omega \subset \mathbb{C}^m$, $c, c^* \in \Omega$ *(these hypothesises are only intuitions and are yet to be proven)* and

$$f(z_1, \ldots, z_m) = \sum_{\alpha_1=0}^{+\infty} \sum_{\alpha_2=0}^{+\infty} \cdots \sum_{\alpha_m=0}^{+\infty} a_{\alpha_1,\alpha_2,\ldots,\alpha_m} (z_1 - c_1)^{\alpha_1} (z_2 - c_2)^{\alpha_2} \ldots (z_m - c_m)^{\alpha_m}$$

on some polydisk centered at $c = (c_1, \ldots c_m)$ and included in $\Omega$. Then there exists a non-empty polydisk $\mathcal{D} \subset \Omega$ centered at $c^* = (c_1^*, \ldots, c_2^*)$ such that

$$\forall z \in \mathcal{D}, f(z_1, \ldots, z_m) = \sum_{\alpha_1=0}^{+\infty} \sum_{\alpha_2=0}^{+\infty} \cdots \sum_{\alpha_m=0}^{+\infty} a^*_{\alpha_1,\alpha_2,\ldots,\alpha_m} (z_1 - c_1^*)^{\alpha_1} (z_2 - c_2^*)^{\alpha_2} \ldots (z_m - c_m^*)^{\alpha_m}$$

And for any of these polydisks, we have $\forall (\alpha_1, \alpha_2, \ldots, \alpha_m) \in \mathbb{N}^m$,

$$a^*_{\alpha_1,\ldots,\alpha_m} = \sum_{k_1=\alpha_1}^{+\infty} \sum_{k_2=\alpha_2}^{+\infty} \cdots \sum_{k_m=\alpha_m}^{+\infty} \binom{k_1}{\alpha_1} \cdots \binom{k_m}{\alpha_m} a_{k_1,\ldots,k_m} (c_1^* - c_1)^{k_1-\alpha_1} \ldots (c_m^* - c_m)^{k_m-\alpha_m}$$

*(The formula is however at least true when $c*$ is inside a polydisk centered at $c$ that is included in $\Omega$)*

As mentioned before, this way of computing the coefficients causes a loss of precision, as the coefficients expressions must be truncated. Furthermore, in the above formula, coefficients of a given order are computed using only coefficients of the orders above. Consequently, the loss of precision is more important on coefficients of higher orders. To compute these coefficients up to a given order N, the referenced `PowerSeries` coefficients are first computed up to order N, and then every coefficient's expression is truncated to use only these coefficients. If needed, it is however possible to give an additionnal named argument `trunc_order` to the `compute_coefficients!` method of `TranslatedSeries` to compute the coefficients of the referenced series up to this `trunc_order` and then use these additionnal coefficients to compute the `TranslatedSeries`' coefficients up to order N.

## 2.4 PDESeries and LocalizedPDESeries

`PDESeries` and `LocalizedPDESeries` allow the user to generate series as a result of well-posed partial differential equations.

### 2.4.1 The "self" series

To construct a series out of a partial differential equation, one has to first write down the partial differential equation, and the unknown for which it has to be solved. This unknown is defined with the help of some particular `ScalarSeriesSymbol` objects. `ScalarSeriesSymbol` usually refer to a `PowerSeries`. However, it is possible to have them refer to a `Symbol` instead. This is used to create `ScalarSeriesSymbol` that refer to a series that has not been created yet. In this case, the `Symbol` used is `:self`, which indicates that the series exists by itself (see ScalarSeriesSymbol for more details)

It is possible to create `ScalarSeriesSymbol` or an `ArrayScalarSeriesSymbol` by using the function `selfseries_symbols`: `selfseries_symbols()` will generate a `ScalarSeriesSymbol` that refers to series `:self` while `selfseries_symbols`$(n_1, n_2, \ldots, n_D)$ will generate an `ArrayScalarSeriesSymbol, D` of size $n_1 \times n_2 \times \cdots \times n_D$ of that refer to series `:self`

### 2.4.2 Writing the partial differential equation

Writing the partial differential equations, with its boundary conditions is done using `SymbolicSeries`, for which a number of operations have been defined (multiplication, addition, substraction, equality, ...) to represent operations on power series. Each `SymbolicSeries` represents a **scalar** power series, making it possible to use Julia native array operations to write multiple equations at once.

`SymbolicSeries`, or `ArraySymbolicSeries` can be constructed in several ways:

- `SymbolicSeries(sss::ScalarSeriesSymbol, center::Vector)` – creates a `SymbolicSeries` from a ScalarSeriesSymbol, around a given `center`
- `SymbolicSeries(sss::Array<:ScalarSeriesSymbol, center::Vector)` – creates an `ArraySymbolicSeries` around the same `center`
- `SymbolicSeries(ps::PowerSeries)` – creates either a `SymbolicSeries` or an `ArraySymbolicSeries` depending on the size of the `ps` from a given `PowerSeries`.
- `SymbolicSeries{D}(x::Number)` – creates a constant `SymbolicSeries` of `D` variables. This can be useful when creating an `Array` of `SymbolicSeries` from `SymbolicSeries` and constants at the same time.

To write equations, several operations are implemented between `SymbolicSeries` and/or constants (`SymbolicSeries{D}` represents a `SymbolicSeries` of `D` variables):

| operation | first argument | second argument |
|---|---|---|
| + | `SymbolicSeries{D}`<br>`SymbolicSeries{D}`<br>`SymbolicSeries{D}` | `SymbolicSeries{D}`<br>`Number`<br>`SymbolicSeries{0}` |
| - | `SymbolicSeries{D}`<br>`SymbolicSeries`<br>`SymbolicSeries`<br>`SymbolicSeries` | `SymbolicSeries{D}`<br>`Number`<br>`SymbolicSeries{0}`<br>———— |
| * | `SymbolicSeries`<br>`SymbolicSeries{D}` | `Number`<br>`SymbolicSeries{D}` |
| / | `SymbolicSeries` | `Number` |

Details on the implementation can be found in the Appendix

All operations +, -, * involving a `SymbolicSeries` and a constant (either `Number` or `SymbolicSeries{0}`) can be used the other way around.

This set of operations is however quite limited. To access a wider range of operations, one needs to be able to evaluate `SymbolicSeries` in some variables and / or constants. If s is a `SymbolicSeriesD`, this can simply be done using `s(`$x_1, x_2, \ldots, x_D$`)` where $x_1, x_2, \ldots, x_D$ can either be variables (obtained with the `@variables` macro) or constants.

Once this is done, one obtains an `EvaluatedSymbolicSeries` and a number of additionnal operations:

| operation | first argument | second argument |
|---|---|---|
| $+$ | `EvaluatedSymbolicSeries{D}`<br>`EvaluatedSymbolicSeries{D}`<br>`EvaluatedSymbolicSeries{D}`<br>`EvaluatedSymbolicSeries{D}`<br>`EvaluatedSymbolicSeries{D}` | `EvaluatedSymbolicSeries{D}`<br>`SymbolicSeries{D}`<br>`Number`<br>`SymbolicSeries{0}`<br>`EvaluatedSymbolicSeries{0}` |
| - | `EvaluatedSymbolicSeries{D}`<br>`EvaluatedSymbolicSeries{D}`<br>`EvaluatedSymbolicSeries{D}`<br>`EvaluatedSymbolicSeries{D}`<br>`EvaluatedSymbolicSeries{D}`<br>`EvaluatedSymbolicSeries` | `EvaluatedSymbolicSeries{D}`<br>`SymbolicSeries{D}`<br>`Number`<br>`SymbolicSeries{0}`<br>`EvaluatedSymbolicSeries{0}`<br>——— |
| $*$ | `EvaluatedSymbolicSeries`<br>`EvaluatedSymbolicSeries` | `Number`<br>`EvaluatedSymbolicSeries` |
| $/$ | `EvaluatedSymbolicSeries` | `Number` |
| $\partial_x$ where $\partial_x$ is a `Differential` object from `Symbolics` | `EvaluatedSymbolicSeries` where the variable of the `Differential` is among the series variables<br>`Array<:EvaluatedSymbolicSeries` | ———<br>——— |
| $\int$ | `EvaluatedSymbolicSeries`<br>`EvaluatedSymbolicSeries` | $x$, a variable of the series<br>$a$, $b$, $x$ where $a$ is the lower bound, $b$ the upper bound (Allowed arguments are the same as those allowed for evaluating a SymbolicSeries) and $x$ a variable of the series |
| ~ | `EvaluatedSymbolicSeries`<br>`EvaluatedSymbolicSeries` | `EvaluatedSymbolicSeries`<br>`Number` |

Details on the implementation can be found in the Appendix.

Once again, if it makes sense mathematically, arguments can be used in the other order.

The operator `~` is used to create `SymbolicSeriesEquation`, i.e equations between series. That can then be used to construct `PDESeries` and `LocalizedPDESeries`

### 2.4.3 An important note on the use of operators with series that do not have the same variables or centers

The available operations on SymbolicSeries make this module quite flexible. However, the user should be aware that operators `+`, `-`, `*`, and `~` are not completely symmetric, especially when used with series that do not share the same variables or centers. Here is how this case is handled:

- First, if the series do not have the same variables, the missing variables are added to each series. This operation is symmetric.
- Secondly, if the variables are not in the same order, the variables of the right term are put in the same order as the variables of the left term. While not symmetric, this operation does not introduce any algorithmic complexity
- Thirdly, if the centers do not match, unspecified components (that may come from variables padding) are set to the other series' components. This again is symmetric

- Lastly, if the centers still do not match, the center of the second series is translated to the center of the first series. This is not symmetric and introduces algorithmic complexity, as well as the need to use LocalizedPDESeries instead of PDESeries. When this happens, a warning is triggered. You can read Translation of a SymbolicSeries for details on the implemntation

The translation phase might make it impossible to compute the coefficients, due to how equations between the series' coefficients are inferred, especially when differential operators are present. Here is an example of an equation and how it is handled:

```
PDE = D2x(K(x,y)) - D2y(K(x,y)) ~ K(x,y)*(l(y)+c)/e
```

Here, `K` is centered at $(0, 1)$ and `l` is centered at 0. `D2x` and `D2y` represent the differential operators at order 2 in $x$ and $y$.

For the left hand side of the equation, everything has the same variables, in the same order, and the same center. Thus the resulting left hand side is a series in variables $(x, y)$ and centered at $(0, 1)$.

For the right hand side, $\frac{l(y)+c}{e}$ is a series in $y$, centered at 0. Thus, it is first transformed to a series in $(y, x)$ that has center $(0, : unspecified)$. Then since the order of variables does not match $K$, it is transformed into a series in $(x, y)$, and center $(: unspecified, 0)$. Finally, the unspecified center component is matched with $K$ and it becomes centered at $(0, 1)$.

In the end, the `~` operator does not need any reordering of variables or translation so everything works fine.

Now let's imagine that the equation was written as

```
PDE = (l(y)+c)/e * K(x,y) ~ D2x(K(x,y)) - D2y(K(x,y))
```

instead

Then for the left hand side, $\frac{l(y)+c}{e}$ is first padded to become a series that has variables $(y, x)$ and centered at $(0, : unspecified)$. Then, K variables are inverted so it becomes a series in $(y, x)$ and centered at $(1, 0)$. Then the unspecified center component of $\frac{l(y)+c}{e}$ is replaced with 0 and finally $K$ is translated to be centered around $(0, 0)$.

This time, when the `~` operator is called, it first has to reorder the variables of the right hand side, and then translate it to $(0, 0)$. This means that the right hand side ultimately depends on every coefficients of `D2x(K(x,y)) - D2y(K(x,y))` and since there are second derivatives, for a given computation order `N`, the right hand side will depend on every coefficients of `K` up to order `N+2`, which will not only make it impossible to compute the coefficients up to order `N` since coefficients of order up to `N+2` necessarily appear, but will also fail the equation inference logic, which will discard all these equations since they have coefficients of order strictly greater than `N` (see The compute_coefficients! method for more details), ultimately resulting in solver failure.

To avoid these problems, always put the series in the expected final state to the left of these operators. Furthermore, **translating differentiated unknown series will not work**. If really needed, one should try to integrate the equation, and give the `compute_coefficients!` method the additionnal `maxIntegrationOrders` argument.

### 2.4.4 Constructing PDESeries and LocalizedPDESeries

The constructors for the `PDESeries` and `LocalizedPDESeries` are exactly the same. They have the following arguments:

- `{T}` – The type parameter of the coefficients' type
- `seriesID::Symbol` – The series' ID
- `variables::VectorNum` – The variables of the series
- `center::Vector` – The center of the series
- `unknown::ScalarSeriesSymbol` or `unknown::Array{<:ScalarSeriesSymbol}` – The `ScalarSeriesSymbol` referring to `:self` series that were used in the equations
- `equations::Vector{<:SymbolicSeriesEquation}` – The partial differential equation(s) and its boundary conditions

Additionnaly, if the unknown series in your equations is integrated, you will need to provide an additionnal argument: `maxIntegrationOrders::Vector{Int}`. For each equation, indicate how many times the unknown series has been integrated at most. This will be used to deduce which equations should be solved when computing the coefficients.

The main difference between `PDESeries` and `LocalizedPDESeries` comes from the algorithmic comlexity of the coefficients' computation, and when they can be used: Some operations, such as evaluating a `SymbolicSeries` at the wrong points or variables, or summing two series that do not have the same center component, or integrating might make the resulting series' coefficients depend on an infinite number of coefficients. This implies that to be computed numerically, the coefficients expressions will have to be truncated up to a certain order. In this case, `LocalizedPDESeries` should be used.

Otherwise, `PDESeries` can be used. When using `PDESeries`, coefficients can be computed with their exact value order by order, this results in the algorithmic complexity being $O(N \times N^D)$ where $N$ is the order up to which the coefficients are computed and $D$ the number of variables.

When computing the coefficients of a `LocalizedPDESeries`, only approximations of the coefficients can be obtained and they all have to be computed at the same time, thus resulting in an algorithmic complexity of $O(N^{2\times D})$. Thus, whenever possible, one should use `PDESeries` rather than `LocalizedPDESeries`.

Both `compute_coefficients!` method offer two named arguments: `verbose`, an integer between 0 and 3 that is 0 by default and `solver` that corresponds to `LinearSolve`' default solver by default but can be replaced by any other solver from this module. Note that for these two `PowerSeries`, the type parameter `T` is used to convert the coefficients of the `LinearProblem` that has to be solved to the type `T`. Thus if one wants to use a GPU solver for instance, it is possible to choose `T=Float32` to make the computation faster (at the cost of precision).

Finally, note that computing the coefficients of the `PDESeries` will make the `SeriesCoefficient` stored in the unknown `ScalarSeriesSymbol` change so that they refer to the `PDESeries` instead of `:self`. Thus, avoid using the same unknown `ScalarSeriesSymbol` multiple times if you don't want that. (See ScalarSeriesSymbol, SeriesCoefficient and The compute_coefficients! method for more details)

### 2.4.5 An example

Here is a full example using `PDESeries`:

```julia

# variables and differential operators
@variables x y
Dx, Dy = Differential(x), Differential(y)

# parameters
q = 1
N = 30

E_ps = TaylorExpansionSeries{Float64}(:E, [x], [-1.2-x^3;0;;0;1.5+x^2], [0])
compute_coefficients!(E_ps, N+1);
E = SymbolicSeries(E_ps)

C_ps = TaylorExpansionSeries{Float64}(:C, [x],
    [3*cos(x);1+2*exp(x);;sin(2*x);1/(3+x^2)], [0])
compute_coefficients!(C_ps, N)
C = SymbolicSeries(C_ps)
C0 = SymbolicSeries{1}[C[1,1];0;;0;C[2,2]]

# unknowns
unknowns = selfseries_symbols(2,2)
K = SymbolicSeries(unknowns, [0,0])

# PDEs and boundary conditions
PDEs = E(x)*Dx(K(x,y)) + Dy(K(x,y))*E(y) .~ K(x,y)*(C(y)-Dy(E(y))) - C0(x)*K(x,y)
BC1 = K(x,x)*E(x) - E(x)*K(x,x) .~ C(x) - C0(x)
BC2 = K(x,0)*E(0)*[q, 1] .~ 0

# PDESeries and computing the coefficients
K_ps = PDESeries{Float64}(:K, [x,y], [0,0], unknowns, [BC1...; BC2...; PDEs...;])
compute_coefficients!(K_ps, N)

```

# 3 Detailed documentation

The objective of this part of the documentation is to explain how the program is constructed to give useful insight to adapt it to other uses. One can find more informations on every function and type defined in this project in the documentation of the code itself. Users who want to implement new functionalities are encouraged to check the existing utilitary functions in the "utilitaries.jl" file

## 3.1 PowerSeries

### 3.1.1 PowerSeries attributes

`PowerSeries{T,D}` is the main object of this module. Its interface requires several attributes to be defined, as well as the `compute_coefficients!` method:

- `T` – The type the coefficients will be stored as
- `D` – The number of dimensions of the series (similar to `Array`). `D` must be greater or equal to 1
- `seriesID` – A unique ID used to identify the series
- `size::NTupleD, Int` – The dimensions of the series (similar to `Array`)

- `variables::VectorNum` – The variables of the series (can be obtained using `@variables`)
- `center::Vector` – The center of the series, Must be the same length as `variables`
- `coefficients::Array{Vector{T},D}` – The series' already computed coefficients. The index of the array corresponds to a scalar series and the coefficients are then stored inside a vector, using linear indexing (see Coefficients indexing : lin, trunc, fullsym for details)
- `order::Int` – The order up to which the coefficients have already been computed. Usually, when none has been computed yet, the order is defined as -1
- `scalar_series_ref::ArrayAbstractScalarSeriesSymbol, D` – For each scalar series, a `ScalarSeriesSymbol` representing it (see ScalarSeriesSymbol for details)
- `compute_coefficients!(ps::PowerSeries, N::Int)` – A method to compute the coefficients of the series up to order N, store them in the `coefficients` array and update the `order`

Additionnaly, `AbstractPowerSeries{D}` is an abstract type representing a `PowerSeries{T,D}` for any type `T`.

The `Base.show` method is implemented for `PowerSeries`

### 3.1.2 ScalarSeriesSymbol

Some `PowerSeries` may require access to a number of informations on a series' coefficient, such as a `Num` representing it, its value if it has already been computed, its order...

These informations are stored inside a `SeriesCoefficient` (see SeriesCoefficient) and these `SeriesCoefficient` are stored inside `ScalarSeriesSymbol`. For this purpose, `ScalarSeriesSymbol` implement the `getindex` method: Using the truncated indexing (see Coefficients indexing : lin, trunc, fullsym), one can use this method to create a `SeriesCoefficient` representing the scalar series coefficient if it has not yet been created and then access it. If the `SeriesCoefficient` has already been created, then the `getindex` method simply allows to access it.

One can access the `ScalarSeriesSymbol` of a `PowerSeries` using the `getindex` method on the `PowerSeries`. The `ScalarSeriesSymbol` attributes are the following:

- `ps::UnionNothing, PowerSeries, Symbol` – a reference to the `PowerSeries` it represents. Additionnaly, it can be set to `nothing` when the series has not yet been initialized or `:self` to reference the self series.
- `scalar_idx::Tuple` – The scalar index of the `ScalarSeriesSymbol` inside the ps. Doing ps[scalar_idx...] returns the `ScalarSeriesSymbol` itself
- `coefficients::DictVector, SeriesCoefficient` – A dictionary where the `SeriesCoefficient` are stored. Its keys are vectors of indices (using the truncated convention)

Additionnaly, the function `selfseries_symbols(size::VarargInt)` can be used to generate an array of `ScalarSeriesSymbol` that refer to series `:self` (or a `ScalarSeriesSymbol` if `size` is empty)

The `Base.show` method is implented for `ScalarSeriesSymbol`

`AbstractScalarSeriesSymbol` is defined to allow the definition of `PowerSeries`' `scalar_series_ref` attribute.

### 3.1.3 SeriesCoefficient

The `SeriesCoefficient{D}` represents a power series' coefficient that may or may not have been computed yet. Its attributes are the following:

- `D` – The number of dimensions of the `PowerSeries` it relates to
- `ps::UnionAbstractPowerSeriesD, Symbol` – The `PowerSeries` it relates to
- `sym::Num` – A symbol representing the coefficient. Probably disgarded, should not be used
- `unique_sym::Num` – A unique Symbolics' representation of the coefficient, automatically created when using the `SeriesCoefficient` constructor. The `unique_sym` has the following format: `var"$(sc.ps.seriesID)$(sc.index)$(sc.indices)"` where `sc` is the `SeriesCoefficient`
- `indices::VectorInt` – The indices of the coefficient in the scalar series (using the truncated convention)
- `index::NTupleInt, D` – The index of the scalar series inside the `PowerSeries`

Using `sc.ps[sc.index...][sc.indices...]` where `sc` is a `SeriesCoefficient` will return `sc`

The `getindex` method of `ScalarSeriesSymbol` should be preferred as a way to create a `SeriesCoefficient`. However, it is possible to create one using the default constructor:

```
SeriesCoefficient(ps::Union{AbstractPowerSeries{D}, Symbol},
                  sym::Num,
                  indices::Vector{Int},
                  index::NTuple{D, Int})
```

`getOrder(sc::SeriesCoefficient)` returns the order of the `SeriesCoefficient`

The `Base.show` method is implemented for `SeriesCoefficient`

### 3.1.4 Coefficients indexing : lin, trunc, fullsym

When working with power series, several conventions are possible to index the coefficients. In this program, three different ones are used, depending on the context:

- linear (lin) – The series is represented as

  $$K(x,y,z) = a_1 + a_2x + a_3y + a_4z + a_5x^2 + a_6xy + a_7xz + a_8y^2 + a_9yz + a_{10}z^2 + \dots$$

  Here, the indices are $1, 2, 3, 4, \dots$. This is convention used when the coefficients have to be stored in a single vector, such as for the `coefficients` attribute of `PowerSeries`

- truncated (trunc) – The series is represented as

  $$\begin{aligned} K(x,y,z) &= a_{0,0,0} + a_{1,0,0}x + a_{1,1,0}y + a_{1,1,1}z + a_{2,0,0}x^2 + a_{2,1,0}xy \\ &\quad + a_{2,1,1}xz + a_{2,2,0}y^2 + a_{2,2,1}yz + a_{2,2,2}z^2 + \dots \\ &= \sum_{i=0}^{+\infty} \sum_{j=0}^{i} \sum_{k=0}^{j} a_{i,j,k} x^{i-j} y^{j-k} z^k \end{aligned}$$

  This convention allows to easily access the order of the coefficient, as it is always the first index. Thus it allows for easy truncation of a series up to a given order. This is the convention usually used to access a `PowerSeries`' `SeriesCoefficient`

- fully symetric (fullsym) – The series is represented as

$$\begin{aligned}K(x,y,z) &= a_{0,0,0} + a_{1,0,0}x + a_{0,1,0}y + a_{0,0,1}z + a_{2,0,0}x^2 + a_{1,1,0}xy \\ &+ a_{1,0,1}xz + a_{0,2,0}y^2 + a_{0,1,1}yz + a_{0,0,2}z^2 + \dots \\ &= \sum_{i=0}^{+\infty}\sum_{j=0}^{+\infty}\sum_{k=0}^{+\infty} a_{i,j,k}x^i y^j z^k\end{aligned}$$

  This is the most natural convention, and it is indeed the most practical when performing operations on power series. Thus, it is the convention chosen for `SymbolicSeries` indexing.

Several functions, defined in the "utilitaries.jl" file allow to translate indices from one convetion to another. They all take a `Vararg` as an argument and return either a `Vector` when the result is truncated or fully symetric and an `Int` when the result is linear. These functions are the following:

- `convertIndices_trunc_to_lin`
- `convertIndices_fullsym_to_lin`
- `convertIndices_fullsym_to_trunc`
- `convertIndices_trunc_to_fullsym`

## 3.2 Series defined by Partial Differential Equations

Since power series can possibly have an infinite number of terms, it is not possible to easily represent operations on them numerically. To do so, a first approach is to truncate the series. This is what is done in the TaylorSeries module [5]. However, since `PowerSeries` are supposed to represent power series algebraically, this is not the approach that what chosen here (even though the TaylorSeries module is used to compute `TaylorExpansionSeries`' coefficients).

Instead, operations on power series are represented as a tree of operations. The nodes of this tree are `SymbolicSeries` or `EvaluatedSymbolicSeries` and its leaves are `ScalarSeriesSymbol`. This way, the truncation can be applied only when computing the coefficients.

### 3.2.1 SymbolicSeries, EvaluatedSymbolicSeries and SymbolicSeriesEquation

`SymbolicSeries{D}` are the leaves of the tree of operations on power series. Its attributes are the following:

- `D` – The number of variables of the `SymbolicSeries`
- `ref::Union{ScalarSeriesSymbol, NlinearSeriesOperation, MultilinearSeriesOperation}` – Either a `ScalarSeriesSymbol` if the `SymbolicSeries` is a leaf, or an operations on other `SymbolicSeries` (see NlinearSeriesOperation and MultilinearSeriesOperation for details)
- `center::Vector` – The center of the series. `center` components can be set to `:unspecified` if not specific (for instance for constant series). In this case, it will be changed to the other series' center component when performing an operation with another series that evaluates in the same corresponding variable
- `get_selfseries_coefficients` – A function that, given a `Vararg{Int, D}`, returns the set of `SeriesCoefficient` referring to `:self` that appear in the operation. This is used by the `compute_coefficients!` method. This function has a named optionnal argument `N` which is the truncation order to be passed if an operation implies an infinite number of coefficients

An example is provided at the end of NlinearSeriesOperation section.

Since some operations require the series involved to be evaluated in some variables, the `EvaluatedSymbolicSeries{D}` can do just that. Its attributes are:

- `D` – The number of variables of the series
- `series::SymbolicSeries{D}` – The series to evaluate
- `variables::Vector{Num}` – The variables in which the series is being evaluated. Has length D.

Finally, `SymbolicSeriesEquationD` represents an equation between two `SymbolicSeries`. It has two attributes:

- `LHS::Union{SymbolicSeries{D}, Number}` – The left hand side
- `RHS::Union{SymbolicSeries{D}, Number}` – The right hand side

The `Base.show` method is implemented for `SymbolicSeries`, `EvaluatedSymbolicSeries` and `SymbolicSeriesEquation`, even though it may not give any particular insights to display those.

### 3.2.2 NlinearSeriesOperation

`NlinearSeriesOperation` are used to represent a linear operation between a finite number of terms that is known before choosing a coefficient.

Their attributes are the following:

- `func` – The function used to aggregate all the arguments. It should take a `Vector` as input
- `args::Vector` – The arguments of the function. These can be `SymbolicSeries`, numbers, or some symbols: `:idxi` represents the i-th index of the series' coefficient while `:∞` represents positive infinity and should be replaced by the truncation order `N` when computing the operation.

Here is an example of how it is used to implement the "+" operator:

```
function Base.:+(s1::SymbolicSeries{D}, s2::SymbolicSeries{D}) where D

    # computing the new center
    center = []
    for (c1, c2) in zip(s1.center, s2.center)
        if c1 != :unspecified && c2 != :unspecified && c1!=c2
            throw(ArgumentError("Can only add SymbolicSeries that have the same
            ↪  centers"))
        elseif c1 == :unspecified
            push!(center, c2)
        else
            push!(center, c1)
        end
    end

    # write the "+" operator as a NlinearSeriesOperation between two series
    op = NlinearSeriesOperation(v -> v[1] + v[2], [s1, s2])

    # define the new SymbolicSeries get_self_series_coefficient
    get_selfseries_coefficients(I::Vararg{Int, D}; N=nothing) =
        union(s1.get_selfseries_coefficients(I...; N),
```

```
                 s2.get_selfseries_coefficients(I...; N))

        # construct the new SymbolicSeries
        SymbolicSeries(op, center,  get_selfseries_coefficients)

    end
```

### 3.2.3 MultilinearSeriesOperation

`MultilinearSeriesOperation` are used to represent a linear operation between a possibly non finite number of terms, that can only be determined once a coefficient is chosen

Their attributes are the following:

- `func` – The function used to aggregate all the arguments. It should take a `Vector` as input
- `arg` – A function which, when called on some indices, will return an argument to be aggregated using `func`. This function must return either a `SeriesCoefficient`, `Number`, or an operation (`NlinearSeriesOperation`, `MultilinearSeriesOperation`) on series coefficients and numbers. This function **cannot** return anything representing a series. It should also accept an named argument `params::Vector` which by default can be set to `[]`.
- `arg_parameters::Vector` – A `Vector` of parameters to pass to the `arg` function. These parameters can be `SymbolicSeries`, `Number`, or similar to `NlinearSeriesOperation`, special `Symbols` such as `idxi` (where i is an integer between 1 and the number of variables of the series it is used inside), `:∞`
- `ranges::Vector{Tuple{Any,Any}}` – The ranges between which the indices to be passed to `arg` vary. Can be integers but also symbols `:idxi`, `:∞`

When using this `MultilinearSeriesOperation` (see getindex method on SymbolicSeries and getNum and getSymbolics method), the `arg` function will first be called on every possible index between `ranges`, and with the optionnal argument `params=arg_parameters` to construct a vector of arguments that will then be passed to the aggregation function `func`

As an example, here is how the multiplication between two `SymbolicSeries` of one variable is implemented:

```
    function Base.:*(s1::SymbolicSeries{1}, s2::SymbolicSeries{1})

        # construct the new center
        center = []
        for (c1, c2) in zip(s1.center, s2.center)
            if c1 != :unspecified && c2 != :unspecified && c1!=c2
                throw(ArgumentError("Can only multiply SymbolicSeries that have the
                ↪  same centers"))
            elseif c1 == :unspecified
                push!(center, c2)
            else
                push!(center, c1)
            end
        end

        # construct the arg function that returns an operation between two
        # of the series coefficients (indexing a SymbolicSeries is
```

```
        # explained in the next section)
        arg(j; params::Vector=[]) =
            NlinearSeriesOperation(v -> v[1]*v[2],
                                   [s1[j,N=params[2]],
                                    s2[params[1]-j, N=params[2]]])

        # create the MultilinearSeriesOperation
        op = MultilinearSeriesOperation(x -> Base.:+(x...), # aggregation
                                        arg, # arg
                                        [:idx1,:oo], # parameters (*)
                                        [(0,:idx1)]) # range

        get_selfseries_coefficients(i::Int; N=nothing) =
            union([union(s1.get_selfseries_coefficients(j;N),
                         s2.get_selfseries_coefficients(i-j;N)
                        ) for j in 0:i]...)

        # construct the resulting SymbolicSeries
        SymbolicSeries(op, center, get_selfseries_coefficients)
    end
```

(*) *`:oo` is `:∞` where ∞ can be obtained by typing \infty*

### 3.2.4 getindex method on SymbolicSeries

Operations on power series ultimately translate to operations on their coefficients. Thus, one needs to access a `SymbolicSeries` "coefficients". This is the purpose of the `getindex` method as defined on `SymbolicSeries`. Once it has been used, the `SymbolicSeries` tree, which leaves are `ScalarSeriesSymbol` becomes a tree which leaves are `SeriesCoefficient`. This tree can then be passed to other functions that can explore it and developp the operations such as getNum and getSymbolics method.

Here are the details of the `getindex`:

```

function Base.getindex(s::SymbolicSeries{D}, args::Vararg{Any,D}; N=nothing) where D
# optional named argument N is the truncation order

    # truncation ==> return 0 if the coefficient order is above N
    if !(isnothing(N)) && D > 0 && +(args...) > N
        return 0
    end

    #= substitution:
        - :idxi is replaced with args[i]
        - :oo is replaced with N
    =#
    idx_subst = Dict{Symbol, Union{Int, Nothing}}([Symbol("idx$i") => v for (i,v) in
    ↪   enumerate(args)])
    idx_subst[:oo] = N

    # substitution on tuples (used for ranges)
    substitution2(t) = (substitution(t[1]), substitution(t[2]))

    function substitution(arg)
        if arg isa ScalarSeriesSymbol
            # ScalarSeriesSymbol becomes a SeriesCoefficient
```

```
            arg[convertIndices_fullsym_to_trunc(args...)...]
        elseif arg isa SymbolicSeries
            # Apply getindex to contained SymbolicSeries
            arg[args..., N=N]
        elseif arg isa NlinearSeriesOperation
            # substitute in every argument of NlinearSeriesOperation
            NlinearSeriesOperation(arg.func, map(substitution, arg.args))
        elseif arg isa MultilinearSeriesOperation
            # substitute in every arg_parameter and range
            MultilinearSeriesOperation(arg.func,
                                       arg.arg,
                                       substitution.(arg.arg_parameters),
                                       substitution2.(arg.ranges))
        elseif arg in keys(idx_subst)
            # substitute :idxi and :oo
            idx_subst[arg]
        else
            # if something else is present, do nothing
            arg
        end
    end

    # apply substitution to the series' ref attribute
    substitution(s.ref)
end
```

A `getindex` method with the same signature also exists for `EvaluatedSymbolicSeries` and will simply apply `getindex` to its `series` attribute.

### 3.2.5 getNum and getSymbolics method

Once a tree of operations that contains only finite numbers of coefficients has been constructed, it is possible to explore it to get some results from it. The `getNum` and `getSymbolics` methods allow the user to retrieve Symbolics' representation of these operations. The only difference between the two come from how `SeriesCoefficient` objects are handled.

`getSymbolics(::SeriesCoefficient)` will simply return the `SeriesCoefficient`' `unique_sym` attribute while `getNum(::SeriesCoefficient)` will return its value if it has already been computed, or its `unique_sym` if it refers to series `:self` (and throw an error otherwise).

Otherwise, here is how these methods behave when applied to different objects (with the example of `getNum`):

- `x::Number` – returns `x`
- `::Nothing` – returns `nothing`
- `::NlinearSeriesOperation` – Applies `getNum` to every argument, aggregates the results and then returns `getNum` applied to the result of the aggregation
- `::MultilinearSeriesOperation` – Applies `getNum` to every result of `arg` within range, aggregates the results and then returns `getNum` applied to the result of the aggregation

`getNum` and `getSymbolics` are also available for `SymbolicSeries`, `EvaluatedSymbolicSeries` and `SymbolicSeriesEquation`, and in those cases, take extra arguments to indicate which indices the `getindex` method should be called with before applying `getNum` and `getSymbolics`

#### 3.2.6 The compute_coefficients! method

With the previous `getNum` method, it is possible to get equations between series' coefficients. But it is not the only action, the `compute_coefficients!` method has to perform. Here is the detail of how it behaves

- First, it generates all the unknown `SeriesCoefficient` that should appear in the equations
- Then it generates all the possible equations with this pattern:
  1. For a given `SymbolicSeriesEquation`, generate all the indices it should be called at up to N plus the corresponding maximum integration order.
  2. For each index, use the `get_selfseries_coefficients` method to get all the unknown coefficients that appear
  3. If no unknown series' coefficient appear, or coefficients of order higher than N appear, then the equation is disgarded
  4. Otherwise, the equation is generated using `getNum` and stored for later
- Once all the equations have been generated, a `LinearProblem` is created from these equations and then solved by the solver
- Room is made to store the new coefficients
- The new coefficients are stored in the `PowerSeries`' `coefficients` attribute.
- If the `PowerSeries` is a `PDESeries`, the coefficients that previously refered to `:self`, that have now been computed are changed to now refer to the `PDESeries`. This is done to be able to compute the coefficients order by order.
- The order is updated

In the case of `PDESeries`, the indices corresponding to the equations used to compute the coefficients of previous orders are also discarded.

# 4 Appendix

In this appendix one will find the mathematical development behind every operation on `SymbolicSeries` and `EvaluatedSymbolicSeries`

## 4.1 Evaluating a SymbolicSeries

Let $K(x_1, \dots, x_D) = \sum_{i_1=0}^{+\infty} \cdots \sum_{i_D=0}^{+\infty} K_{i_1,\dots,i_D} (x_1 - c_1)^{i_1} \dots (x_D - c_D)^{i_D}$, and let's say that one wants to evaluate $K$ in some variables or constants $y_1, ..., y_D$. Let's compute the expressions of the resulting series' coefficients.

For that, let's first rewrite

$$K(x_1, \dots, x_D) = \sum_{i_1=0}^{+\infty} \cdots \sum_{i_{D-1}=0}^{+\infty} (x_1 - c_1)^{i_1} \dots (x_{D-1} - c_{D-1})^{i_{D-1}} \sum_{i_D=0}^{+\infty} K_{i_1,\dots i_D} (x_D - c_D)^{i_D}$$

Then we have several possibilities:

#### 4.1.1 $y_D = c_D$

In that case, the result is a series of $D-1$ variables, evaluated in $y_1, \dots, y_{D-1}$ for which the coefficients are $K_{i_1,\dots,i_{D-1},0}$:

$$K(y_1, \dots, y_D) = \sum_{i_1=0}^{+\infty} \cdots \sum_{i_{D-1}=0}^{+\infty} K_{i_1,\dots,i_{D-1},0}(y_1 - c_1)^{i_1} \dots (y_{D-1} - c_{D-1})^{i_{D-1}}$$

One can then proceed to evaluate $y_{D-1}$

#### 4.1.2 $y_D \neq c_D$ is a constant

In that case, the result is a series of $D-1$ variables, evaluated in $y_1, \dots, y_{D-1}$ for which the coefficients are $\sum_{i_D=0}^{+\infty} K_{i_1,\dots,i_{D-1},i_D}(y_D - c_D)^{i_D}$ (The use of `LocalizedPDESeries` is required):

$$K(y_1, \dots, y_D) = \sum_{i_1=0}^{+\infty} \cdots \sum_{i_{D-1}=0}^{+\infty} \left( \sum_{i_D=0}^{+\infty} K_{i_1,\dots,i_{D-1},i_D}(y_D - c_D)^{i_D} \right) (y_1 - c_1)^{i_1} \dots (y_{D-1} - c_{D-1})^{i_{D-1}}$$

One can then proceed to evaluate $y_{D-1}$

#### 4.1.3 $y_D$ is a variable and $y_D \notin \{y_1, \dots, y_{D-1}\}$

In that case one can rearrange the sum and proceed to evaluate $y_{D-1}$

#### 4.1.4 $y_D$ is a variable and $\exists k \in \{1, \dots, D-1\}, y_k = y_D$

For the sake of simplicity, let's assume that $k = D - 1$. one then has

$$K(y_1, \dots, y_D) = \sum_{i_1=0}^{+\infty} \cdots \sum_{i_{D-1}=0}^{+\infty} (y_1 - c_1)^{i_1} \dots (y_{D-1} - c_{D-1})^{i_{D-1}} \sum_{i_D=0}^{+\infty} K_{i_1,\dots i_D}(y_{D-1} - c_D)^{i_D}$$

$(y_{D-1} - c_D)^{i_D}$ can then be rewritten as

$$(y_{D-1} - c_D)^{i_D} = (y_{D-1} - c_{D-1} + c_{D-1} - c_D)^{i_D} = \sum_{k=0}^{i_D} \binom{i_D}{k} (y_{D-1} - c_{D-1})^k (c_{D-1} - c_D)^{i_D - k}$$

Thus

$$
\begin{aligned}
\sum_{i_{D-1}=0}^{+\infty} \sum_{i_D=0}^{+\infty} & K_{i_1,\dots i_D} (y_{D-1} - c_{D-1})^{i_{D-1}} (y_{D-1} - c_D)^{i_D} \\
&= \sum_{i_D=0}^{+\infty} \sum_{k=0}^{i_D} \sum_{i_{D-1}=0}^{+\infty} \binom{i_D}{k} K_{i_1,\dots i_D} (y_{D-1} - c_{D-1})^{k+i_{D-1}} (c_{D-1} - c_D)^{i_D-k} \\
&= \sum_{i_D=0}^{+\infty} \sum_{k=0}^{i_D} \sum_{i_{D-1}=k}^{+\infty} \binom{i_D}{k} K_{i_1,\dots i_D} (y_{D-1} - c_{D-1})^{i_{D-1}} (c_{D-1} - c_D)^{i_D-k} \\
&= \sum_{i_D=0}^{+\infty} \sum_{i_{D-1}=0}^{+\infty} \sum_{k=0}^{min(i_{D-1},i_D)} \binom{i_D}{k} K_{i_1,\dots i_D} (y_{D-1} - c_{D-1})^{i_{D-1}} (c_{D-1} - c_D)^{i_D-k} \\
&= \sum_{i_{D-1}=0}^{+\infty} \left( \sum_{i_D=0}^{+\infty} \sum_{k=0}^{min(i_{D-1},i_D)} \binom{i_D}{k} K_{i_1,\dots i_D} (c_{D-1} - c_D)^{i_D-k} \right) (y_{D-1} - c_{D-1})^{i_{D-1}}
\end{aligned}
$$

Once again, the use of `LocalizedPDESeries` is required. One then proceeds to evaluate $y_{D-1}$

## 4.2 Operations defined on SymbolicSeries

The operations $+$, -, $/$ and multiplication by a scalar for `SymbolicSeries` that have the same centers are straight forward.

We present here how translation of a series in case one tries to do an operation between two series that do not have the same centers happen, and how multiplication of two series is implemented

### 4.2.1 Translation of a SymbolicSeries

For the sake of simplicity, let us focus on translation of a series of one variable:

In this case, let's say that we want the center $c$ to become $c'$. We have

$$
\begin{aligned}
f(x) = \sum_{i=0}^{+\infty} f_i (x-c)^i \\
&= \sum_{i=0}^{+\infty} f_i (x - c' + c' - c)^i \\
&= \sum_{i=0}^{+\infty} f_i \sum_{k=0}^{i} \binom{i}{k} (x-c')^k (c'-c)^{i-k} \\
&= \sum_{k=0}^{+\infty} \left( \sum_{i=k}^{+\infty} \binom{i}{k} f_i (c'-c)^{i-k} \right) (x-c')^k
\end{aligned}
$$

For a series of multiple variables, one simply multiplies the binomial coefficients and powers of the center components difference. i.e :

$$
f(x_1, \dots, x_D) = \sum_{i_1=0}^{+\infty} \cdots \sum_{i_D=0}^{+\infty} \sum_{k_1=i_1}^{+\infty} \cdots \sum_{k_D=i_D}^{+\infty} \binom{i_1}{k_1} \cdots \binom{i_D}{k_D} f_{i_1,\dots,i_D} (c'_1 - c_1)^{i_1-k_1} \dots (c'_D - c_D)^{i_D-k_D}
$$

Notice that the resulting series' coefficients depend on an infinite number of the original series' coefficients and will thus require the use of LocalizedPDESeries rather than PDESeries.

This translation operation is implemented with the `translate` function.

### 4.2.2 Multiplication of two SymbolicSeries

Let $F(x_1, \ldots, x_M) = \sum_{i_1=0}^{+\infty} \cdots \sum_{i_M=0}^{+\infty} F_{i_1,\ldots,i_M}(x_1 - c_1)^{i_1} \ldots (x_M - c_M)^{i_M}$ and
$G(y_1, \ldots, y_N) = \sum_{j_1=0}^{+\infty} \cdots \sum_{j_N=0}^{+\infty} G_{j_1,\ldots,j_N}(y_1 - c'_1)^{j_1} \ldots (y_N - c'_N)^{j_N}$

Let's denote $\{z_1, \ldots, z_D\} = \{x_1, \ldots, x_M\} \cap \{y_1, \ldots, y_N\}$ and assume that
$\{x_{D+1}, \ldots, x_M\} = \{x_1, \ldots, x_M\} \setminus \{y_1, \ldots, y_N\}$ and that
$\{y_{D+1}, \ldots y_N\} = \{y_1, \ldots, y_N\} \setminus \{x_1, \ldots, x_M\}$ (once again, for the sake of simplicity)

Then this implies that $\forall i \in \{1, D\}, c_i = c'_i$. Thus,

$$F(x_1, \ldots, x_M)G(y_1, \ldots, y_N) =$$
$$\sum_{i_{D+1}=0}^{+\infty} \cdots \sum_{i_M=0}^{+\infty} \sum_{j_{D+1}=0}^{+\infty} \cdots \sum_{j_N=0}^{+\infty} (A)\,(x_{D+1} - c_{D+1})^{i_{D+1}} \ldots (x_M - c_M)^{i_M} (y_{D+1} - c'_{D+1})^{j_{D+1}} \ldots (y_N - c'_N)^{j_N}$$

where

$$A = \left(\sum_{i_1=0}^{+\infty} \cdots \sum_{i_D=0}^{+\infty} F_{i_1,\ldots i_M}(z_1 - c_1)^{i_1} \ldots (z_D - c_D)^{i_D}\right) \left(\sum_{j_1=0}^{+\infty} \cdots \sum_{j_D=0}^{+\infty} G_{j_1,\ldots j_N}(z_1 - c_1)^{j_1} \ldots (z_D - c_D)^{j_D}\right)$$
$$= \sum_{i_1=0}^{+\infty} \cdots \sum_{i_{D-1}=0}^{+\infty} (z_1 - c_1)^{i_1} \ldots (z_{D-1} - c_{D-1})^{i_{D-1}} \left(\sum_{i_D=0}^{+\infty} F_{i_1,\ldots,i_M}(z_D - c_D)^{i_D}\right) \times$$
$$\left(\sum_{j_D=0}^{+\infty} \left(\sum_{j_1=0}^{+\infty} \cdots \sum_{j_{D-1}=0}^{+\infty} G_{j_1,\ldots,j_N}(z_1 - c_1)^{j_1} \ldots (z_{D-1} - c_{D-1})^{j_{D-1}}\right) (z_D - c_D)^{j_D}\right)$$
$$= \sum_{i_1=0}^{+\infty} \cdots \sum_{i_{D-1}=0}^{+\infty} (z_1 - c_1)^{i_1} \ldots (z_{D-1} - c_{D-1})^{i_{D-1}} \times$$
$$\sum_{i_D=0}^{+\infty} \left(\sum_{j_D=0}^{i_D} F_{i_1,\ldots,j_D,\ldots,i_M} \left(\sum_{j_1=0}^{+\infty} \cdots \sum_{j_{D-1}=0}^{+\infty} G_{j_1,\ldots,i_D-j_D,\ldots,j_N}(z_1 - c_1)^{j_1} \ldots (z_{D-1} - c_{D-1})^{j_{D-1}}\right)\right) (z_D - c_D)^{i_D}$$
$$= \sum_{i_D=0}^{+\infty} (z_D - c_D)^{i_D} \sum_{j_D=0}^{i_D} \left(\sum_{i_1=0}^{+\infty} \cdots \sum_{i_{D-1}=0}^{+\infty} F_{i_1,\ldots,j_D,\ldots,i_M}(z_1 - c_1)^{i_1} \ldots (z_{D-1} - c_{D-1})^{i_{D-1}}\right) \times$$
$$\left(\sum_{j_1=0}^{+\infty} \cdots \sum_{j_{D-1}=0}^{+\infty} G_{j_1,\ldots,i_D-j_D,\ldots,j_N}(z_1 - c_1)^{j-1} \ldots (z_{D-1} - c_{D-1})^{j_{D-1}}\right)$$
$$\ldots$$
$$= \sum_{i_1=0}^{+\infty} \cdots \sum_{i_D=0}^{+\infty} \sum_{j_1=0}^{i_1} \cdots \sum_{j_D=0}^{i_D} F_{j_1,\ldots,j_D,i_{D+1},\ldots,i_M} \times G_{i_1-j_1,\ldots,i_D-j_D,j_{D+1},\ldots,j_N} \times (z_1 - c_1)^{i_1} \ldots (z_D - c_D)^{i_D}$$

Thus, the coefficients of the resulting series are

$$\sum_{j_1=0}^{i_1} \cdots \sum_{j_D=0}^{i_D} F_{j_1,\ldots,j_D,i_{D+1},\ldots,i_M} \times G_{i_1-j_1,\ldots,i_D-j_D,j_{D+1},\ldots,j_N}$$

### 4.3 Operations defined on EvaluatedSymbolicSeries

Operators +, -, /, ~and multiplication are straight forward for `EvaluatedSymbolicSeries`. Here is the explanation of how differentiation and integration are handled.

Additionnaly, if one wants to perform operations on series which do not have the same variables, or not in the same order, it is possible and these functionalities are handled by the `union_variables` and `swap_variables` functions respectively (and are used by default by the operators presented here)

#### 4.3.1 Differentiation of an EvaluatedSymbolicSeries

Let $F(x_1, \dots, x_D) = \sum_{i_1=0}^{+\infty} \sum_{i_D=0}^{+\infty} F_{i_1,\dots,i_D}(x_1 - c_1)^{i_1} \dots (x_D - c_D)^{i_D}$

Then

$$\frac{\partial F}{\partial x_j}(x_1, \dots, x_D) = \sum_{i_1=0}^{+\infty} \cdots \sum_{i_D=0}^{+\infty} (i_j + 1) F_{i_1,\dots,i_j+1,\dots,i_D}(x_1 - c_1)^{i_1} \dots (x_D - c_D)^{i_D}$$

#### 4.3.2 Integration of EvaluatedSymbolicSeries

The integration with respect to one of the series' variables is defined as

$$\int_{c_j}^{x_j} F(x_1, \dots, y, \dots, x_D) dy = \sum_{i_1=0}^{+\infty} \cdots \sum_{i_j=1}^{+\infty} \cdots \sum_{i_D=0}^{+\infty} \frac{1}{i_j} F_{i_1,\dots,i_j-1,\dots,i_D}(x_1 - c_1)^{i_1} \dots (x_D - c_D)^{i_D}$$

It is then possible to define

$$\int_a^b F(x_1, \dots, x_j, \dots, x_D) dx_j = G(x_1, \dots, b, \dots, x_D) - G(x_1, \dots, a, \dots, x_D)$$

where $G(x_1, \dots, x_j, \dots, x_D) = \int_{c_j}^{x_j} F(x_1, \dots, y, \dots, x_D) dy$

## References


[1] M. Krstic and A. Smyshlyaev, *Boundary Control of PDEs: A Course on Backstepping Designs*, SIAM, Philadelphia, 2008.

[2] R. Vazquez, G. Chen, J. Qiao, and M. Krstic, "The power series method to compute backstepping kernel gains: Theory and practice," in *Proc. 63rd IEEE Conference on Decision and Control (CDC)*, 2024.

[3] J.-M. Coron, R. Vazquez, M. Krstic, and G. Bastin, "Local exponential $H^2$ stabilization of a $2 \times 2$ quasilinear hyperbolic system using backstepping," *SIAM J. Control Optim.*, vol. 51, pp. 2005–2035, 2013.

[4] Jiří Lebl, "Tasty Bits of Several Complex Variables, a whirlwind tour of the subject"

[5] Luis Benet, David P. Sanders, "A Julia package for Taylor polynomial expansions in one or more independent variables.", `https://github.com/JuliaDiff/TaylorSeries.jl`